

Ideas

Single Player

- 1. BarFight
- 2. Maze Escape

Other

- 1. Special Chess

The Following Up of Ideas

Special Chess

Principal

Classic chess game with additional pieces and powerups.

Requirements

Must Include	Should Include	Could Include
A classic chess game and board	On board powerups	Guide
Additional pieces	Display of possible moves for a piece	Play against bot
Settings menu to enable or disable pieces		

Setting

Chess

Maze Escape

Principal

Two dimensional procedurally generated maze with a timer.

Requirements

Must Include	Should Include	Could Include
A maze	Procedural generation	Darkness around player to hide full maze
A timer	Difficulty settings to control maze size	
An exit portal		

Setting

None

BarFight

Principal

A two dimensional retro boxing game, similar to Mortal Combat and Half Sword.

Requirements

Must Include	Should Include	Could Include
Bots to play against	Different attacks	Interactive Environment/Props
Bar like setting	Multiple enemies	Alcohol
Stamina bar	Consecutive hit counter	Powerups
Hit delay		Impact frames
Small stun upon receiving hit		
Health		

Setting

A dimly lit put in a small village. It should feel quite cramped and dingy.
In the assumption that this is done in Greenfoot, certain features may prove harder to

implement. Thus this document seeks to preemptively address difficulties.

Potential Features List

Must Include	Should Include	Could Include
Bots to play against	Different attacks	Interactive Environment/Props
Bar like setting	Multiple enemies	Alcohol
Stamina bar	Consecutive hit counter	Powerups
Hit delay		Impact frames
Small Stun upon receiving hit		
Health		

Bots

The easiest thing to do for this, or the bare minimum, would be to have them swing randomly.

This could be extended by making them **react on certain conditions**, such as the relative distance of the player. This could be expanded on by having these methods and attributes in a `bot` class. It would have limiting attributes such as **reaction time**. This would run a for loop to represent hesitation.

The most extreme fulfilment of this idea would be to use libraries to train the ai. This may not be possible due to the limitations of Greenfoot.

Stamina Bar

As something for both players, this would be an attribute stored in a fighter class, housing **both the player and the bot**.

The most reasonable method of implementation for this would be to impose an **ability cost** to each. For example performing an uppercut may cost an arbitrary 10 `stamina`.

Stun

This would be similar to the **hesitation loop** from the bot.

The easy implementation would be identical to stamina control, with each attack adding to a `player_stun` attribute.

In extending this, multiple attacks in combination could continue to increase this stun number to a threshold. This would hinge on the condition of `time_since_hit > X`. Then the loop would commence, stunning the player.

Punch

This would be a significant challenge due to animating in Greenfoot. Not only this, but it should have collision. My idea for this is that it would come in two parts: one player animation, and one created projectile with collision. To avoid the activating player from also colliding with this, I would have a class for attacks, split by **Player** and **Bot**. The movement for this "attack" object largely depends on the intended complexity. It could simply move 5 units in a straight line and then disappear. The alternative is to have it loosely follow gravity, posing its velocity as a vector such as $10\mathbf{i} - 2\mathbf{j}$.

Analysis

My current idea for this project is for it to provide a simple, retro-themed fighting game for one player. It will involve using different moves, each controlled by appropriate key bindings, to tackle various opponents with equal power. Opponents would be generated based on the player's options with extra attributes such as reaction time to make it fair. Further scaling of the project would include a difficulty slider or even different rounds fighting enemies of greater and greater strength or speed. This would challenge the user, making the game fun and engaging to play.

My current knowledge of the games industry shows that similar games are underrepresented, despite their undisputed popularity amongst all audiences. Current examples include Street Fighter and Mortal Kombat. Other games with desirable features for this product are Gladio Mori, Half Sword and Watermelon Sandbox. Whilst two of these are 3d, I admire their menus, theming and core mechanics. This project has the goal of filling this niche in a scalable manor. This design could remove the problem of boredom from the user as they would have never truly beaten it.

The embarkment of this project excites me as I can appreciate the dedication and creativity in the games industry. By completing this relatively small project, I would be able to use it for developing my skills. An ideal development engine should be incredibly minimal, thus allowing me to use all of my skills organically, rather than having them built in. Additionally, it ought to be compatible across PC operating systems such as MacOS, Debian and NixOS. This would garner a wider player base and expand the clientele of the game.

For this project to be successful, it should allow a player to fight against a condition based bot. The player should have to balance various displayed attributes such as stamina and health whilst beating their opponent. Examples of the bot's conditions may include the proximity of the player, the stamina of the player or the health of the bot.

Research



Qualities to Take Forwards	Qualities I Dislike
I like the clear UI displaying the health of each fighter, this makes it easy to quickly gauge the opponent's strength mid-fight. Consequently, the game is easier to read, meaning that it could be more accessible to an external audience. This opens up streaming as a viable form of marketing.	I'm not a fan of the difficult colour scheme, making it difficult to interpret. This hinders the user's ability to interpret the given information. Furthermore, it might harm the ability for visually challenged people to enjoy any content produced with it.
I like the pixelated graphics since they add to the character of the game. This makes the style seem authentic and more likeable from an audience or consumer.	But I think that mine should have greater clarity to blend in with similar and more modern games and feel closer to a remake. By doing this I could engage prior audiences of this media.



Qualities to Take Forwards	Qualities I Dislike
I think the menu of Half Sword sets up the theme of the game perfectly before the game is properly started. This is great because it clearly informs the player of the genre and style.	I think that in a minimal environment I would struggle to replicate this menu. It would be needlessly complicated and in depth than the game itself.
The combat in the game is incredibly realistic with conditionally actioning bots. I really like this because it greatly simplifies the concepts and code of the bots. This is good for my minimal IDE where training a full AI may not be possible.	Again, the combat is too detailed. It would be far too much to add realistic bruising and effects to a simple 2d fighter.



Qualities to Take Forwards	Qualities I Dislike
The simplified models for player and enemies would make development of my game much easier. It would also standardise player hitboxes and sizes. This would be ideal for my game because a faster development process would make for a longer amount spent on implementation of desired features, which is the core idea of this genre.	In my final product, I aim to properly sprite the characters. This would make it more engaging for the user as it would be easier to relate to. Games like Mario show that players are warmer to a game featuring humanoid characters and so mine should do similar.
The idea of multiple enemies is intriguing. It would veritably add depth to further difficulties in the game. By having multiple enemies onscreen concurrently, it would take longer for the player to win the fight. This addition could be an option for users wishing for a longer experience.	It would be difficult to have multiple enemies all in view, or without getting in the way. This could in turn make it slower or jankier to play, reducing the amount of time players are likely to spend playing it.



Qualities to Take Forwards	Qualities I Dislike
I am a big fan of the chevron above the player's head. This would make the game easier to play by reducing the risk of confusion. Resultantly, consumers are able to pick up the game faster and invest more time in it.	I dislike the use of physical weapons. I am to make my game at least plausible, which the addition of blades would jeopardise. My characters will use hand to hand combat, making it seem much grittier and so appealing to teenage audiences.
The cohesion between the ground and background leads to a seemingly better designed product. This has been known to increase user acceptance rates and so I will aim to have the same.	The apparent monetary system displayed would not fit my game. Were I to implement it, there would not, at current state of planning, be many options of use. Thus I have decided that it is an unnecessary feature which causes conflict with the core mechanics of the game.
I think that the idea of multiple enemies can add a layer of complexity to the game, supported by the zoom of the camera. This added complexity can easily be a worthwhile extension to the game.	The camera has a wide zoom which would not fit the cosy aesthetic of my game. It would be harder to achieve the desired effect of my game.

Stakeholders

My stake holders would likely be teenagers interested in computer science, between 14 and 19. This is because my research and experience shows that this demographic is the most likely to be engaged with similar media. Furthermore, this group tend to be socially active in both real life and online. This level of communication can be capitalised on for easier marketing. Subsequently, I will survey this demographic in order to determine features and implementations which may be successful and desirable in this game.

I believe that my player base will be making greatest use of this product in their free time after education hours. As such, I will aim to make each experience succinct in order to support a user's requirement of homework. Longer games can feel like a greater commitment to play and so are not as popular during this targeted time window.

I aim to include my stake holders in this process in order to maintain a high fidelity to the desired concept and functionality.

My Form Is Linked Below:

[Bar Fight Consumer Research - Fill in form](#)

Consumer Research Findings

On a scale from 1->10, how much would you desire two or more moves?

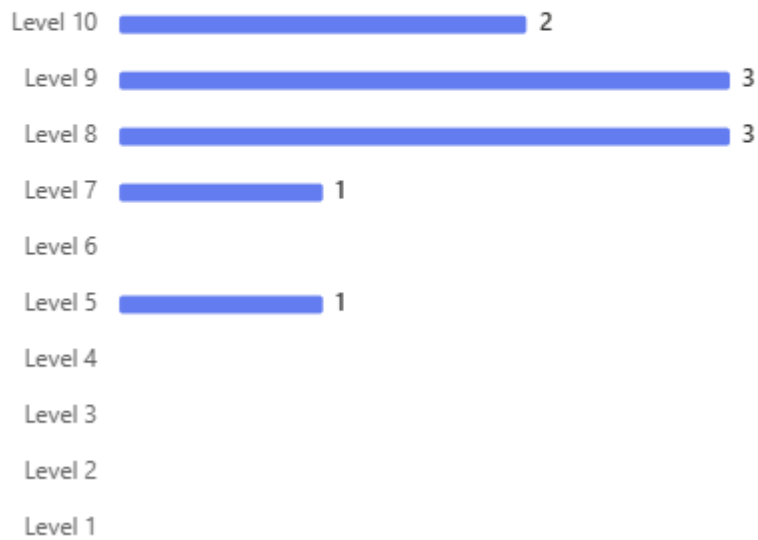


This shows a high demand for the ability to use multiple different attacks. I am grateful for this result since this feature was one which I envisaged in the game, making it fully engaging. Following this result, I shall make this a priority. I can see how this would largely impact the game and make it more exciting to play.

On a scale from 1->10, how much would you desire enemies varying in size and strength?

8.30

Average Rating



As one of the highest rated ideas, this would be a priority for the game. It would make the whole environment feel more alive, with a perceived challenge that could be easily implemented.

On a scale from 1->10, how much would you desire fighting two or more enemies simultaneously?

5.00

Average Rating



Level 10

Level 9

Level 8

Level 7 2

Level 6 3

Level 5 2

Level 4 1

Level 3 1

Level 2

Level 1 1

This shows a fortunate lack of demand for fighting multiple enemies. I can gauge from this that it is far from an the essential feature, and consequentially I shall not list it as a requirement for this game.

On a scale from 1->10, how much would you desire combo moves?

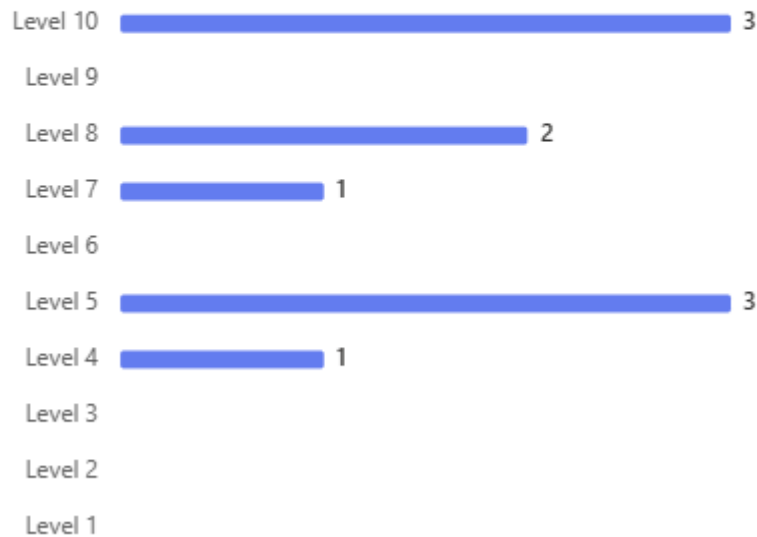


This is one of the most demanded implementations for the game. It would add a necessary skill curve to the game and in turn make it more engaging for both new and advanced players.

- On a scale from 1->10, how much would you desire character customisation?

7.20

Average Rating



As an averagely rated feature, it would not be a priority to implement, but it is worth consideration. The concept of customisation could lead to the addition of more features such as a customisation shop, which is intriguing. Yet it may extend beyond the scope of the project.

On a scale from 1->10, how much would you desire exponential knockback?

7.60

Average Rating

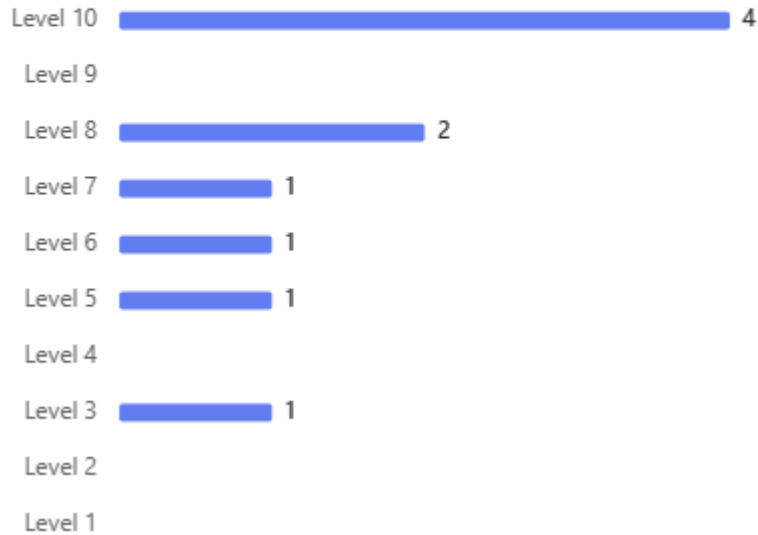


Due to the rating of this feature, and my own personal willingness to implement it, this is likely to make it into the game. It would both make attacks feel more meaningful, whilst also acting as a safety net were the player caught in range of the enemy.

On a scale from 1->10, how much would you desire the use of environmental props?

7.70

Average Rating



As a relatively highly requested addition, this would be very nice to add. It would mesh the background and environment with the player and enemy, creating a worthy cohesion. This would also give the game a unique selling point and further separate it from competition.

On a scale from 1->10, how much would you desire enemies with varying reaction times?



As another vehemently requested feature, this would be an interesting addition to the game, making it feel more challenging and having better scaling. This number could be tweaked also so that the further the player can progress in the game, the harder the enemies become to face.

Subsequent Requirements

	Feature		Sub Features	Explanation	Justification	Importance
1	Combo Moves	A	Varying Attacks With Primary Attack Button	On pressing the given attack button in rapid succession, the first displayed attack would differ to the second/third, which would have a different level of power.	This would encourage longer engagements which could be challengingly risky. This would add a level of difficulty to engage the player. Finally, it is a greatly requested feature and would please stakeholders with its addition.	7/10 (Desirable)
		B	Multiple Inputs for One Attack	Upon entering multiple inputs sequentially,	This adds an element of skill to the game, where more	5/10 (If Possible)

				a different attack is shown with different attributes for knockback, damage, etc.	advanced players can use different abilities. This increases the complexity of the game and thus encourages the player to spend more time playing it.	
	This falls further into desirable content, rather than necessary. Whilst it would surely increase a required skill level of the game, I can see how it might be hard to implement.	C	X	X	X	X
2	Enemies of Varying Sizes and Strengths	A	Random Size	Uses random numbers to scale the displayed enemy within a range fitting of the difficulty.	This shows variance between the enemies and gives the illusion of a larger game. Furthermore, it makes the player anticipate a longer or more difficult fight, engaging them further. Finally, it is a heavily requested feature and would please stakeholders.	8/10 (Desirable)
		B	Random Strength	Uses random numbers to modify the damage done by the enemies' attacks within a range fitting of the difficulty.	This shows variance between the enemies and gives the illusion of a larger game. The random value of the strength could be increased with difficulty, making it more challenging as appropriate.	9/10 (Essential)
	This addition to the game had a vehement popularity from my stakeholders. Furthermore, I am sure that its implementation would coincide	C	Random Reaction Time	Uses random numbers to set the relative reaction time of the enemy for certain conditions.	This would add a further value to change with difficulty. Enemies could react to conditions such as player proximity with	8/10 (Essential)

	pleasantly with other possible features. Hence I shall list this as a required feature.				ease, thus making it harder to safely attack the enemy.	
3	Multiple Different Moves	A	Basic Punch	A simple punch with a small range. This would form the basis of the fighting game.	As the most rudimentary element of any fighting game, this would make it indelibly simple for any new players, whilst also allowing better skilled people to play.	10/10 (Essential)
		B	Kick	A short kick with greater range and damage than the punch.	This might add ideas of decision to the game. It would allow the player to take a gamble and deal more damage, with a greater risk of reciprocating fire or loss of stamina.	9/10 (Essential)
	The basis of this concept has had an overwhelming level of support from my surveyed target audience. This undoubtedly places this in a necessary position.	C	Headbutt	An incredibly short range attack with the highest damage.	The player could use this to punish the enemy, giving an easy solution to close quarters combat. This would then reduce the possibility of a near impossible games for certain difficulties and thus increase user retention rates.	7/10 (Desirable)
4	A Consistent HUD (Heads Up Display)	A	Health Bar	A visible bar above the player's head, designed to indicate a fighter's hit points.	In all of the games which I investigated for this project, this item was prevalent. The reason being that it is an essential indicator of the leading fighter and the proximity to a loss. My use of this would see the traditional red block fill, making use of	10/10 (Essential)

					the user's precognition of its meaning. I feel free to do this due to my target audience having a prior experience of similarly categorised games.	
		B	Stamina Bar	A simple element similar to the health bar, used to show the current level of stamina possessed by the relevant fighter. Positioned below the health bar.	This element would act incredibly like the above. This would show the stamina attribute of the fighter as an orange block fill. As so, it would decrease after the player or enemy made a move, thus showing both the player's capability to advance, or the enemies weakness. By including this as a displayed element, it improves the ease of use for the player. This would differ slightly to the health bar in light of the nature of stamina to be recovered, both in similar media and real life. Hence, mine shall use a formula to determine the rate at which the bar should be filled over a given period. The result would be a smoother looking increase in stamina as opposed to a direct spike.	9/10 (Essential)
	This element would be essential to display current attributes such	C	Attack reset time	A short and semi-permanent addition, appearing after the		4/10 (If Possible)

	as player health and stamina.			player has completed an action with a delay.		
5	Menu Screen	A	A start button	A simple menu item which starts the game on being clicked.	In all three games which I analysed for my market research, a start button was consistent. This feature is useful because it allows the player to instantiate a new game if and when they desire. Furthermore, this permits a period of which a student playing this game can wait before playing again, for example if they are printing out an essay they might load the game when they plan to start printing, but only actually begin it once the document has started to print.	10/10 (Essential)
		B	A difficulty slider	A simple slider produced by having an object follow the mouse when it is in a set region and clicks. Different points on the possible path of this actor would correspond to different ranges for enemy attributes such as damage.	Whilst my survey has not accounted for this idea, I find that nearly all similar games have this feature. Additionally, it would veritably support other favoured features such as enemies with variable reaction times. Hence, I have decided to take agency on this matter and make this a necessary feature in my game. This would be helpful for students playing the game because it would allow them to control	8/10 (Desirable)

					the level of focus required to enjoy the game. Often after a long day, students can struggle to engage with prolonged and intense brain function.	
	A menu screen would be crucial for a typical functionality of the game. Due to the fast paced nature of the game, a menu to break it up would feel helpful. It would also be a necessary platform for elements such a play button.	C	X	X	X	X
6	Movement	A	Movement from left to right	This would enable the player to move their character from left to right with certain inputs. The game would likely use WASD format, as is conventional in this category.	My game would require the ability for the player to move around. This forms the basis for almost all similar games, including all of which I researched prior. I did not include this idea in my survey, due to it being typical.	10/10 (Necessary)
		B				
		C				

Success Criteria			
	Explanation	Justification	Measurability
1A	When the user clicks the mouse in rapid succession, the displayed punch should appear to be from the other arm.	This would make the game feel more realistic due to the way in which boxers vary their strikes for the sake of speed.	It is also measurable, since it is binary in whether it is or isn't implemented.
2B	Randomly generated levels of strength for an enemy fighter.	This is relatively achievable and capable of making the game more realistic and exciting.	It is also measurable, since it is binary in whether

			it is or isn't implemented.
3A	The ability for the fighter to punch or be punched by the enemy.	This is an incredibly basic mechanic, involved in almost all market competitors. It would allow the very basis of a fighting game.	It is also measurable, since it is binary in whether it is or isn't implemented.
3B	The power for the fighter to kick their enemy with greater range and damage to the punch.	This would add a variance to the fighting, with certain moves being best applied under differing circumstances. Furthermore, its implementation would be held back only by a changing of the attributes of it.	It is also measurable, since it is binary in whether it is or isn't implemented.
3C	The allowance of the player to strike their enemy with a headbutt, the attack having greater power and shorter ranges to its peers.	The consequence of this would be the emergence of strategies such as closing distance between user and enemy. This directly adds risk management as a skill to the game, permitting the choice between playstyles.	It is also measurable, since it is binary in whether it is or isn't implemented.
5A	A simple menu screen item being a start button, with the power to start the game upon being clicked.	This allows the user to take agency over the starting of the game. The alternative would force the user to focus immediately after launching the game, where the fight would start as soon as it opens. This would be incredibly difficult and hence shall be thwarted.	It is also measurable, since it is binary in whether it is or isn't implemented.
4A	A health bar to hover over the player or enemies' head and display their remaining health.	This addition to the HUD would allow easy tracking of health metrics, allowing the player to easily gauge the risk of the next exchange.	It is also measurable, since it is binary in whether it is or isn't implemented.
4B	A stamina bar to monitor and show the stamina of the player.	The feature would allow the player to keep track of their stamina. This is useful because this metric decreases on attacks. By keeping it visible, the player can time their attacks and engagements.	It is also measurable, since it is binary in whether it is or isn't implemented.

Possible Limitations		
Description	Justification	Possible Solutions
The animation of characters performing moves such as punches or being hit.	This could be quite difficult as both are situationally dependant. Furthermore, most IDEs ideal for this project such as Greenfoot, have no built in animation feature.	I could possible work around this by changing the character sprites in rapid succession, but this could still prove difficult. Even by doing this, it would still be very difficult to have appropriate reactions.
The smooth refilling of the stamina bar and relevant hiding when full.	This is again down to the issue of animation. In addition, I would like the bar to increment exponentially, yet stop if it depletes further.	I could easily have an exponential increase using a formula such as $\text{increase} = (\text{amount} + 1)^2$ where amount starts as 0 and the above is called after a recovery time.
Initiating the game only upon the start button being pressed in older IDEs such as Greenfoot.	I think that this could be an issue due to the nature of how my enemy would work. I would use a stop function, but this would inhibit any input being detected and thus render the game unplayable.	I could either have two different scenes or blocks of code, or if I were to use Greenfoot, have two separate worlds. This would allow a flexibility that I would otherwise not have, at the cost of not having the same background without interfering.
Stopping and starting the game with a pause button.	Similar to the above, it could be difficult to pause the game and take necessary inputs to affect the state of the game.	I could inhibit player inputs and set a globally accessible variable which would stop the bot from responding to the controlling conditions.

Hardware Requirements

I will need a computer with a keyboard, capable of running Greenfoot version 3.9.0. For the distribution targeted at major Linux based operating systems, a specific CPU is not required. I can justify this choice by assuming that my target demographic uses an OS such as Ubuntu or Debian. This demographic needs the smallest minimum requirement, being only the necessary hardware to interact with the game. A keyboard is necessary to send inputs to the program.

The screen of this device is not of great importance, due to its pixelated graphics.

Software Requirements

The device will need to be running a Debian based system with Oracle JDK pre-installed. As well as this it should have a version of Java capable of interfacing with Greenfoot, my conducted research leads me to believe the recent version would need Java 21. Installing Greenfoot itself is free and works graphically with almost all desktop environments such as KDE Plasma or Gnome. On more advanced environments such as Hyprland, external tools and software may need to be installed for a smoother experience.

Sources

<https://www.greenfoot.org/doc/faq#:~:text=The%20Windows%20version%20of%20Greenfoot,on%20a%2032%2Dbit%20system.>

<https://www.greenfoot.org/download/installation>

https://www.greenfoot.org/version_history

Computational Methods

What Makes This a Computable Task?

This task is computable because it factors in multiple different processes and conditions simultaneously. This would be almost impossible for a human to do. Furthermore, by being run on a computer, the output could be projected to a screen in real time, allowing a better visualisation of the current state of the game.

Abstraction
This project will use abstraction to account for areas such as character hit box, or the size and range of an attack. This will work by casting boxes to account for necessary collision areas, such as a player's body or attack. Resultantly, it will be easier to calculate collisions and if the player is hit. I will be using this because Greenfoot does its own image shape and size calculation which would be harder to understand and detect. My method will also make debugging easier.
Concurrency

My game will use concurrency to facilitate both the player and enemy moving at the same time. This is needed so that the player can play the game with a responsive feel to it. Furthermore, there will be calculations such as damage received or player-enemy proximity happening asynchronously. This extends to the use of hitboxes, player inputs, and enemy condition detection and reaction.

Enumeration

I will use enumeration to list all of the possible states which the player can be in. This would account for the actions the player can take in different conditions, such as if they are stunned or being knocked back. Each state will be set in a usefully accessible location, such that the code for the affected actor can respond accordingly, i.e. temporarily disabling movement actions.

Data Mining

The program will have data mining system inbuilt into its environment. My game would use the Greenfoot built-in method `isKeyDown()` to detect input from the user's keyboard. This data will then be used to follow through with relevant actions and effects. An example could be that by holding down the 'd' or right arrow key, the player would move to the right. I need this for my game so that the player can interact with it.

Decomposition

There will be decomposition in my game, where each problem is broken into smaller and more manageable challenges. By doing so, I will be more efficient at writing code, since I will only have to consider a small region at a time. This also benefits the asynchronous nature of my game, where multiple processes coincide temporally. This idea would be challenged were I not to use decomposition, due to everything being in one large code block.

Bar Fight

Character Control

Player

Input Driven

Enemy

Event Driven

Player Distance
Player Velocity
Player Health
Self Health
Self Stamina

Attacks

Kick

Punch

Left

Right

Animation/
Change Sprite

Jump

Damage

Knockback

Player: Base
Enemy: From Slider

Data Management

Characters

Health
Stamina
Cooldown
Distance
Velocity
Curr. Attack

Environment

Enemy Difficulty
Tick

Enemy Specific

Size
Strength
Speed
Reaction Time

Display

HUD

Pause

Bars

Health

Stamina

Cooldown

Menu

Start Button

Difficulty Slider

Easy

Medium

Hard

Designing

Development Plan

Stage	Involves	Success Criteria	Success Indicator	desirable essential
1	Creating the player.	They should be able to move left, right and up with corresponding input keys. They should also be able to use attacks which briefly change the sprite. The data such as power and knockback of these attacks would not be of concern at this stage.	- [] Player Can move - [] Player can attack - [] Player jumps and returns to ground	essential
2	Creating Enemy	The Enemy actor is added to the scenario with basic movement and abilities to match the player. Its actions such as <code>move</code> should be controlled by monitored events such as Player proximity.	- [] Enemy can move - [] Enemy can attack - [] Enemy responds to Player events	essential
3	Setting up environment data collection.	Greenfoot world contains getters for data such as player health, stamina and distance from enemy. This is so that the enemy class can use these as events to act upon.	- [] Getters return accurate information about Player	essential
4	Creating HUD	The game should now have a HUD to display the player and enemy's health and stamina . The bar should deplete the lower the value represented. The health bars should hover above each characters' head. The stamina and cooldown	- [] Health Bar follows player/enemy - [] Stamina bar in corner - [] Cooldown bar shown when necessary (in use) - [] Bars show accurate	essential with desirable aspects such as floating healthbar

Stage	Involves	Success Criteria	Success Indicator	desirable essential
		bar (unique to player) should sit in the top right hand corner. A pause/play button be added also in the top left to start or stop the game.	information - [] Game can be stopped/started	
5	Start Menu	The game should load into the <code>Start Menu</code> , with options including start the game, change difficulty and quit. Difficulty slider should modify variables in Enemy class such as strength, speed, stamina. Start starts game.	- [] Start button starts game - [] Difficulty slider increases enemy power	essential
6	Better Attacks	Attack functions are populated with data for damage and knockback. For the player this is a base value, for the enemy this is random as above. Damage and knockback are to be taken using setters such that Player can apply knockback to Enemy.	- [] Attack from either lowers health - [] Attack from either has damage proportional knockback	desirable
7	Condition Checking	Basic condition checking such as <code>if(player.health < 0) {player.die;}</code> . Extends to stamina (limit attack), cooldown.	- [] Characters die - [] Attacks remove stamina such that they cannot be used constantly - [] Attacks have effective cooldowns	essential
8	Enemy Expansion	The enemy loads in with random variables within an appropriate range. This includes size (<code>Greenfoot.imageSize</code>), strength (within	- [] Enemy size varies - [] Enemy speed varies - [] Enemy strength varies	desirable

Stage	Involves	Success Criteria	Success Indicator	desirable essential
		bounds relative to difficulty) and speed.		

Justifications

1

This is the most crucial stage, due to it being the crux of the whole game. By doing this first, I will be able to test almost everything else.

2

This stage is necessary for properly wiring the enemy. For testing, this will allow me to test mechanisms such as attacking, character-character collision.

3

By doing this, I can properly wire up the enemy. Furthermore, the data from the getters I plan to implement will serve as useful tests to ensure that everything is going as I'd like. The conditions in the enemy's code coming from here would make the game officially playable.

4

The HUD is the most stylistic element, and while helpful, not crucial to play-ability. This is why I have left this until the characters are set up. As well as this, by leaving this after the 3rd (data collection), all of the methods for returning data to be shown will be available, making this a theoretically smooth experience.

5

Like the HUD, this is not the most important element, hence it has been left to this cycle. I have also positioned it after designing the HUD so that the memory of the necessary code and methods to create a basic GUI will still be fresh in my memory.

6

The better attacks stage plans to populate the attacks with damage and knockback. This enables the stage after it, whilst also testing the health-bar. It does not go directly after the addition of the health-bar, allowing cycle 5 to take that place. This is because of the obvious link between both GUI based stages.

7

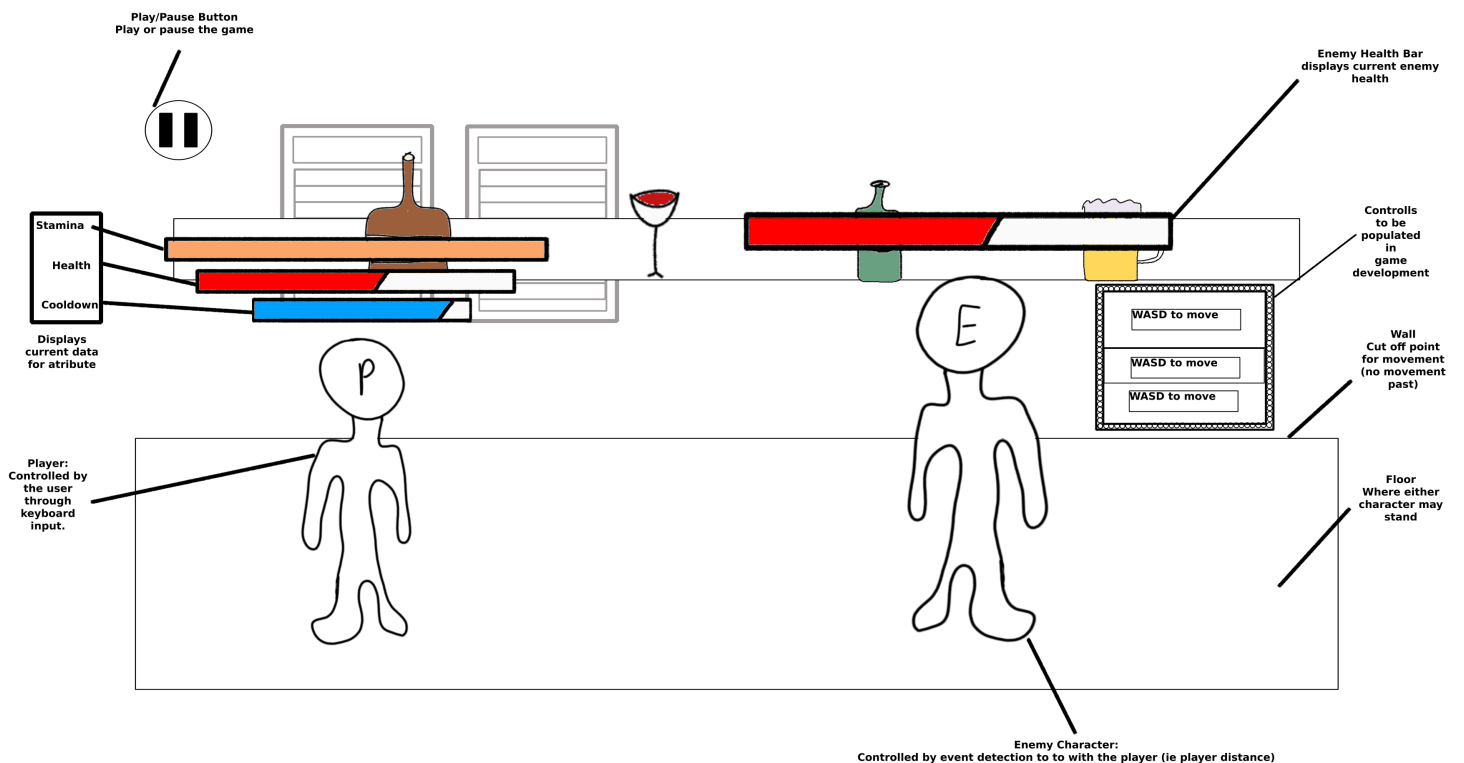
This stage is the final one until the game is entirely functional. It plans to introduce death and a limit on attacks enforced by stamina and cooldown. This stage is being implemented as one of the later stages due to it completing the structure of the game.

As the final stage, it represents a very small and slightly unnecessary aspect of the game, hence its position. This stage would be what I would like, and was lightly desired in my consumer research survey. Unfortunately, it cannot be justified to take precedence over the stages coming before it.

GUI

Main Screen

Bar Fight



The above is a rough outline of what I hope my project might become. As of this, details such as non-important colours including background and player sprites are not shown above.

This screen is the most detailed, because it will be displayed the majority of the time.

Bars

Above each will be coloured bars to display data about metrics corresponding to each character. The amount of the bar coloured in would be the percentage of the metric from full, for example if the player had 40 health remaining down from 100, the health bar would be 40% full.

Colours

The colours for each bar have been chosen to match the aesthetic of my game, whilst taking inspiration from other games of similar genres or categories. As such health would

be red; stamina tan/peach; cooldown blue.

Visibility

As is above, more data would be available for the player's character than for the enemy. This adds more challenge to the game as well as authenticity with the player not knowing some data about their opponent such as stamina.

Furthermore, the cooldown bar would only be temporarily displayed. This being when it is needed (not 0). This would increase the player's view by only showing data useful at that moment.

Size Variance

The larger the value for a character's maximum or starting health, the larger the bar for that should be. In practice this would affect only the enemy once the relevant development cycle is complete.

Play/Pause Button

This button will sit in the top left corner, as my market research has led me to conclude is standard, due to the space freed up on the screen. Upon clicking, it will pause the game and change its sprite to reflect the state. In practice, pausing will look like stopping the movement of either player or enemy and halting the regeneration of stamina or cooldown.

Characters

The game will feature two characters, being the player and the enemy, both visible on the screen. Their movement will be shown live, making the game playable. I plan to draw sprites for each, matching them to the overall look of the game. The examples shown above are rough outlines of what I hope mine become. The size difference between the player and enemy character is intended, and hopefully will reflect plays with a higher difficulty, where the enemy is larger than the player.

Controls Screen

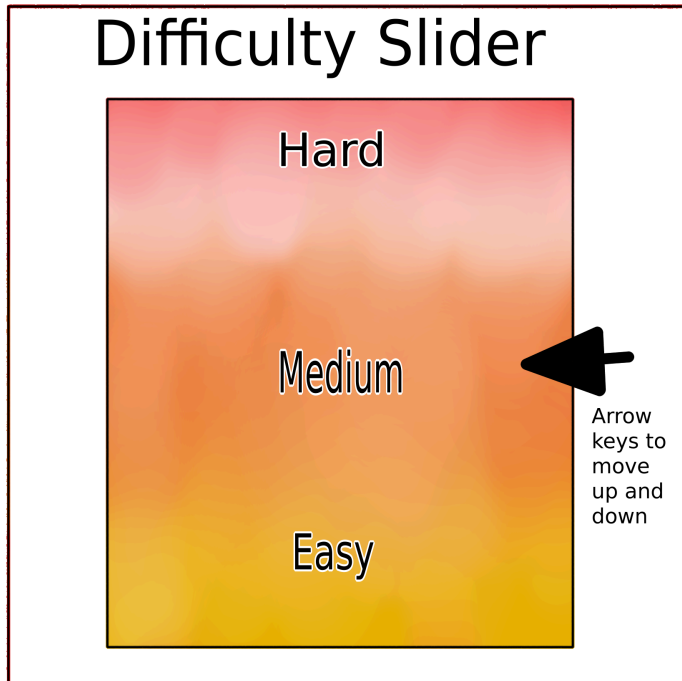
In the background of the scene will be a small display in the style of a bar's menu. It will have on it the various controls to perform basic actions such as move, punch, etc. I have chosen this style because it will reflect the general style of the game. This will increase the continuity of the game, making it feel better tuned and stylised.

Background

The background in the image above is a rough simplification. I plan to keep elements like the bottles, shelves and drinks, but with a more full scene. The scene created by it should feel dark and quite moody, like a small pub.

Start Menu

Bar Fight



Like the main game GUI mock up, this is an abstraction of what I hope the actual thing looks like. Some features above are exaggerated so that they are quicker to find, and more obvious to make. Furthermore, there is no background image, hence the white background.

I have this main menu in order to bridge the gap between loading the game and fighting. This allows the player to spend time selecting a difficulty and preparing themselves.

Difficulty Slider

The difficulty slider above is coloured based on its difficulty. The cursor/selector would be set at medium by default, making the colour system intuitive to the user. The idea of moving the selector with arrow keys may be a little confusing, or not explicitly obvious, so I would like to have a label for this information. I also have deliberately put it on the left, such that the start screen is dominated by itself and the play buttons - the two main features of my start screen. As well as this, the position of the difficulty slider echoes games of a similar category, and the colour scheme would demonstrate this further. The result is a dynamic looking menu, with a clear colour scheme which evidences itself to its genre.

Start Button

The large size of the start button is designed to reflect its significance in the menu. It is on the right because my target audience will likely read from left to right, meaning that they will see the difficulty slider first and respond to that before pressing play. The triangle inside the play button is designed as the general symbol for 'play', in music, video or

games. Due to this being so universal, I will likely remove the label above it. This would further centre it and remove unnecessary detail.

Title

The title sits above everything else, making it the first thing seen. Its audacity is similar to its competitors, thus it should make the game better known. The font above is a placeholder font for what would either be a hand drawn title, or a better font.

Tests Per Stage

Stage 1 - Player Creation

Test ID	Test	Test Data	Outcome	Actual Outcome
1A	Is the player sprite displayed?	Run program	Player sprite should be displayed at the player's location.	
1B	Is the player able to move, corresponding to keyboard input?	Run program	The player moves upon keyboard input. This accurately covers all necessary directions.	
1C	Can the player attack?	Run program	The attack animation is displayed after the relevant keyboard input.	
1D	Does the player fall to the ground after jumping?	Run program	The player falls from the air after jumping, giving the impression of gravity.	
1E	Does the player have values for health,	Run program	Yes, with relevant methods	

Test ID	Test	Test Data	Outcome	Actual Outcome
	stamina and knockback?		to get this data.	

Stage 2 - Enemy Creation

A class called `Enemy` would have been created in order for above stages. This stage populates that class with data, actions and methods.

Test ID	Test	Test Data	Outcome	Actual Outcome
2A	Is the enemy sprite displayed at their location?	Run program	The sprite is displayed wherever the enemy is.	
2B	Is the enemy able to move?	Run program	The enemy can move when commanded with keyboard input.	
2C	Is the enemy able to move in response to data about the player?	Run program	The enemy uses the getter methods from stage 2 to decide when to move.	
2D	Can the enemy attack and decide when to through received data?	Run program	The enemy uses data such as player proximity to "decide" to attack.	
2E	Does the enemy have data for health, stamina and knockback?	Run program	Yes, with methods to return them.	

Stage 3 - Data Collection

Test ID	Test	Test Data	Outcome	Actual Outcome
3A	Do methods for returning data for character attributes such as Health and Stamina return accurate information?	Run program	The methods used to find and return data are accurate.	
3B	Is the method to calculate and return the distance between the player and enemy accurate?	Run program	The method is accurate, returning data in a form which is usable.	

Stage 4 - HUD

Test ID	Test	Test Data	Outcome	Actual Outcome
4A	Does the HUD displays accurate data for the health of either character?	Run program	The HUD displays coloured bars which display either characters' health in real time.	
4B	Does the HUD display a stamina bar for either character, updated in real time?	Run program	The HUD displays coloured bars which display either characters' stamina in real time.	
4C	Does the HUD dynamically	Run program	The HUD displays the player's	

Test ID	Test	Test Data	Outcome	Actual Outcome
	display the player's cooldown as a bar?		cooldown value, in real time, showing only it when necessary (cooldown is not zero).	
4D	Does the HUD display a play/pause button is the top left corner for the user?	Run program	A play/pause button is displayed in the appropriate region. On clicking it the icon/sprite changes to reflect the state, with the game being paused/resumed.	
4E	Do all of the bars show relevant information which is consistently updated?	Run program	Yes.	

Stage 5 - Start Menu

Test ID	Test	Test Data	Outcome	Actual Outcome
5A	Does the game start with a menu?	Run program	The game loads with a menu with the option to start the game.	
5B	Does this start menu include a difficulty slider?	Run program	Yes.	
5C	Does the difficulty slider	Run program	Yes.	

Test ID	Test	Test Data	Outcome	Actual Outcome
5D	accurately update a value which affects the difficulty of the game?			
	Does the start button start the game?	Run program	On click, the start button begins the game with both the player and enemy set up?	

Stage 6 - Better Attacks

Test ID	Test	Test Data	Outcome	Actual Outcome
6A	Do attacks have a value for knockback as well as damage?	Run program	Yes.	
6B	Is knockback applied accurately to either character when necessary?	Run program	Yes.	

Stage 7 - Condition Checks

Test ID	Test	Test Data	Outcome	Actual Outcome
7A	Does the player win	Run program	The player wins the	

Test ID	Test	Test Data	Outcome	Actual Outcome
	when the enemy's health is 0?		game and an appropriate screen is displayed to convey this.	
7B	Does the player lose the game when their health is 0?	Run program	The enemy wins and a message is shown to display this.	
7C	Can the player attack when their stamina is 0?	Run program	The player cannot attack until their stamina is sufficiently regenerated.	
7D	Does the stamina amount regenerate exponentially after a period of inactivity?	Run program	Yes.	
7E	Can the player use an attack when it is on Cooldown ?	Run program	No, they must wait until the Cooldown timer has ended.	
7F	Is a character removed when their health is 0?	Run program	The character is removed from the scene.	

Stage 8 - Better Enemies

Test ID	Test	Test Data	Outcome	Actual Outcome
8A	Does the enemy load with random strength and	Run program	Yes.	

Test ID	Test	Test Data	Outcome	Actual Outcome
8B	knockback in accordance with difficulty?			
	Does the enemy have a random size in accordance with difficulty?	Run program	Yes.	

Final Tests

Test ID	Test	Test Data	Purpose	Actual Outcome
F1	Player Enemy collision	Run program	Tests if the player and enemy are capable of colliding.	
F2	Appropriate knockback levels for attacks	Run program	Tests if the level of knockback applied is suitable for player and enemy.	
F3	Appropriate health values in correspondence with damage	Run program	Tests how long the average game could last and is this is consistent with my analysis and planning.	
F4	The player dying ends the game	Run program	To check if there is an appropriate response to inform the player	

Test ID	Test	Test Data	Purpose	Actual Outcome
			that they have lost.	
F5	The start screen starts the game correctly	Run program	To ensure that random values for the enemy are correctly created with reference to the difficulty slider.	
F6	The gravity of the game works to achieve a pseudo realism	Run program	To see how the game feels to play for a player. A lower gravity could make for a slower game, with more time spent in the air.	

Post Development Questions

Once each of my stages have been implemented, and I have alpha tested, I would like to ask some of my friends and peers to beta test my game. In order for this to be effective and a good use of time, I will put together series of questions to ask them once they have played the game on each difficulty setting.

My questions should aim to cover information about the following:

- User friendliness
- Intuitive design
- Likeability of the design
- Difficulty of each setting
- How long each difficulty took them
- The usability of the controls
- Did the game perform well (high frames per second, etc)

In light of this, my questions will probably look something like this:

1. On a scale of 1 to 10, how user friendly is the game?
2. On a scale of 1 to 10, how intuitive would you describe the design?
3. Did you like the design?
4. On a scale of 1 to 10, how hard was the hardest setting?
5. On a scale of 1 to 10, how hard was the default setting?
6. On a scale of 1 to 10, how hard was the easiest setting?
7. How long did the hardest difficulty take you?
8. How long did the default difficulty take you?
9. How long did the easiest difficulty take you?
10. Did you find the controls to be easy to learn and use?
11. Would you say the game performed well on your system?
12. Did you encounter any bugs/glitches?

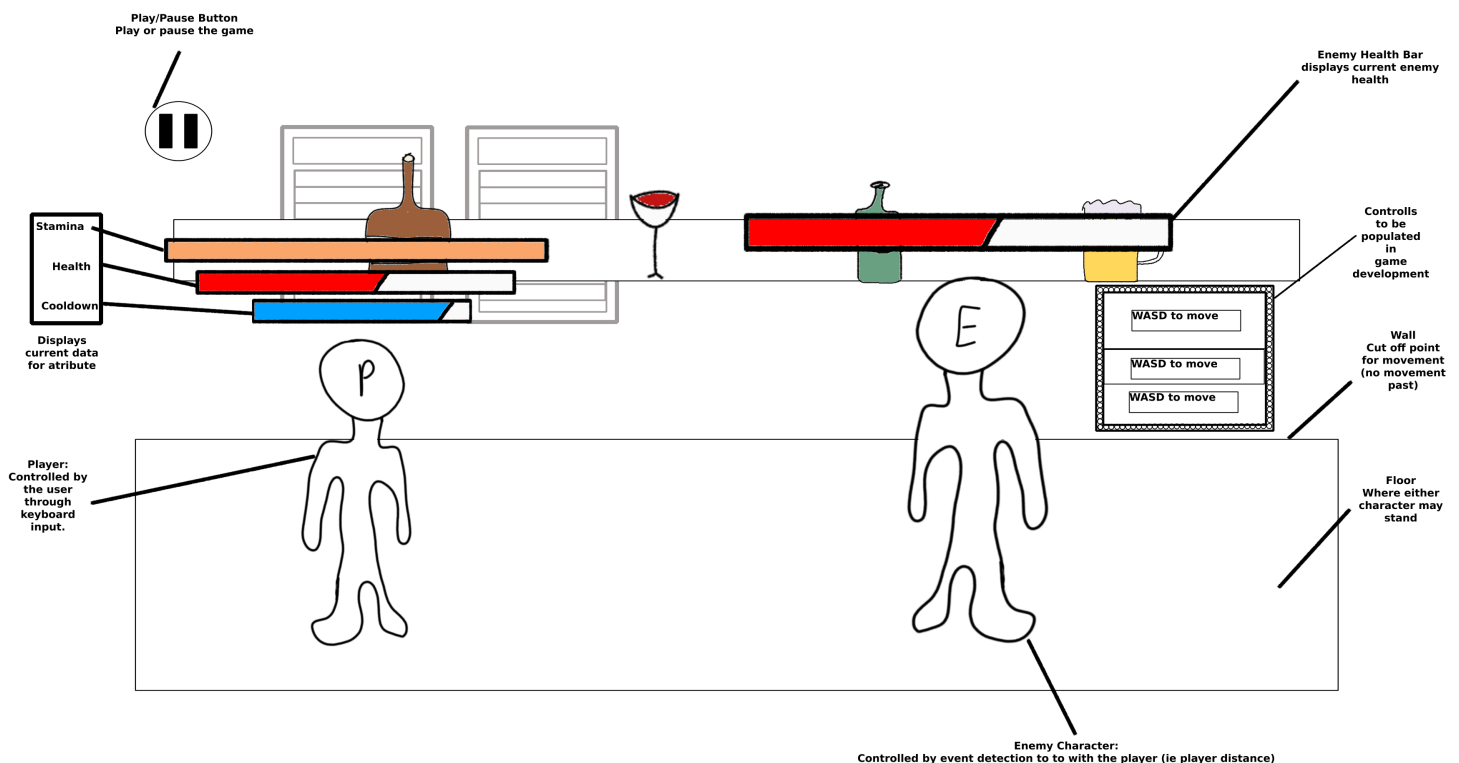
My hope is that the beta testers will enjoy the game and give constructive feedback to work on before it is finished.

This questionnaire will most likely be sent via Microsoft Forms, due to the Microsoft Office Suite being available to my college. This would make it the easiest form of communication with integrated data graphics.

GUI Justifications

Main Screen

Bar Fight



The above is a rough outline of what I hope my project might become. As of this, details such as non-important colours including background and player sprites are not shown

above.

This screen is the most detailed, because it will be displayed the majority of the time.

Bars

Above each will be coloured bars to display data about metrics corresponding to each character. The amount of the bar coloured in would be the percentage of the metric from full, for example if the player had 40 health remaining down from 100, the health bar would be 40% full.

Colours

The colours for each bar have been chosen to match the aesthetic of my game, whilst taking inspiration from other games of similar genres or categories. As such health would be red; stamina tan/peach; cooldown blue.

Visibility

As is above, more data would be available for the player's character than for the enemy. This adds more challenge to the game as well as authenticity with the player not knowing some data about their opponent such as stamina.

Furthermore, the cooldown bar would only be temporarily displayed. This being when it is needed (not 0). This would increase the player's view by only showing data useful at that moment.

Size Variance

The larger the value for a character's maximum or starting health, the larger the bar for that should be. In practice this would affect only the enemy once the relevant development cycle is complete.

Play/Pause Button

This button will sit in the top left corner, as my market research has led me to conclude is standard, due to the space freed up on the screen. Upon clicking, it will pause the game and change its sprite to reflect the state. In practice, pausing will look like stopping the movement of either player or enemy and halting the regeneration of stamina or cooldown.

Characters

The game will feature two characters, being the player and the enemy, both visible on the screen. Their movement will be shown live, making the game playable. I plan to draw sprites for each, matching them to the overall look of the game. The examples shown above are rough outlines of what I hope mine become. The size difference between the player and enemy character is intended, and hopefully will reflect plays with a higher difficulty, where the enemy is larger than the player.

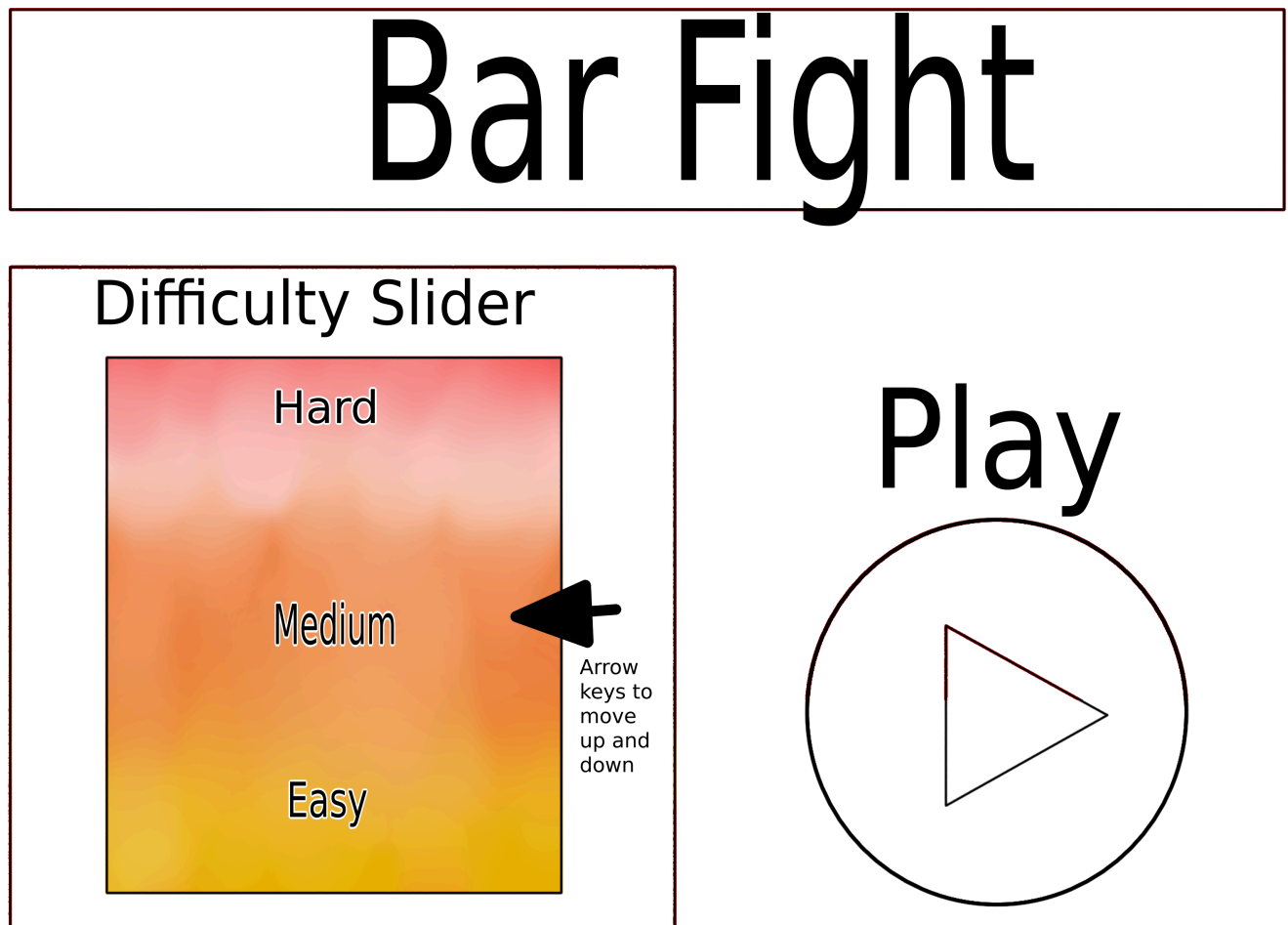
Controls Screen

In the background of the scene will be a small display in the style of a bar's menu. It will have on it the various controls to perform basic actions such as move, punch, etc. I have chosen this style because it will reflect the general style of the game. This will increase the continuity of the game, making it feel better tuned and stylised.

Background

The background in the image above is a rough simplification. I plan to keep elements like the bottles, shelves and drinks, but with a more full scene. The scene created by it should feel dark and quite moody, like a small pub.

Start Menu



Like the main game GUI mock up, this is an abstraction of what I hope the actual thing looks like. Some features above are exaggerated so that they are quicker to find, and more obvious to make. Furthermore, there is no background image, hence the white background.

I have this main menu in order to bridge the gap between loading the game and fighting. This allows the player to spend time selecting a difficulty and preparing themselves.

Difficulty Slider

The difficulty slider above is coloured based on its difficulty. The cursor/selector would be set at medium by default, making the colour system intuitive to the user. The idea of moving the selector with arrow keys may be a little confusing, or not explicitly obvious, so I would like to have a label for this information. I also have deliberately put it on the left, such that the start screen is dominated by itself and the play buttons - the two main

features of my start screen. As well as this, the position of the difficulty slider echoes games of a similar category, and the colour scheme would demonstrate this further. The result is a dynamic looking menu, with a clear colour scheme which evidences itself to its genre.

Start Button

The large size of the start button is designed to reflect its significance in the menu. It is on the right because my target audience will likely read from left to right, meaning that they will see the difficulty slider first and respond to that before pressing play. The triangle inside the play button is designed as the general symbol for 'play', in music, video or games. Due to this being so universal, I will likely remove the label above it. This would further centre it and remove unnecessary detail.

Title

The title sits above everything else, making it the first thing seen. Its audacity is similar to its competitors, thus it should make the game better known. The font above is a placeholder font for what would either be a hand drawn title, or a better font.

Tests

Tests Per Stage

Stage 1 - Player Creation

Test ID	Test	Test Data	Outcome	Actual Outcome
1A	Is the player sprite displayed?	Run program	Player sprite should be displayed at the player's location.	
1B	Is the player able to move, corresponding to keyboard input?	Run program	The player moves upon keyboard input. This accurately covers all necessary directions.	
1C	Can the player attack?	Run program	The attack animation is displayed after the relevant	

Test ID	Test	Test Data	Outcome	Actual Outcome
1D	Does the player fall to the ground after jumping?	Run program	keyboard input. The player falls from the air after jumping, giving the impression of gravity.	
1E	Does the player have values for health, stamina and knockback?	Run program	Yes, with relevant methods to get this data.	

Stage 2 - Enemy Creation

A class called `Enemy` would have been created in order for above stages. This stage populates that class with data, actions and methods.

Test ID	Test	Test Data	Outcome	Actual Outcome
2A	Is the enemy sprite displayed at their location?	Run program	The sprite is displayed wherever the enemy is.	
2B	Is the enemy able to move?	Run program	The enemy can move when commanded with keyboard input.	
2C	Is the enemy able to move in response to data about the player?	Run program	The enemy uses the getter methods from stage 2 to decide when to move.	
2D	Can the enemy attack	Run program	The enemy uses data such as	

Test ID	Test	Test Data	Outcome	Actual Outcome
	and decide when to through received data?		player proximity to "decide" to attack.	
2E	Does the enemy have data for health, stamina and knockback?	Run program	Yes, with methods to return them.	

Stage 3 - Data Collection

Test ID	Test	Test Data	Outcome	Actual Outcome
3A	Do methods for returning data for character attributes such as Health and Stamina return accurate information?	Run program	The methods used to find and return data are accurate.	
3B	Is the method to calculate and return the distance between the player and enemy accurate?	Run program	The method is accurate, returning data in a form which is usable.	

Stage 4 - HUD

Test ID	Test	Test Data	Outcome	Actual Outcome
4A	Does the HUD displays accurate data for the health of either character?	Run program	The HUD displays coloured bars which display either characters' health in real time.	
4B	Does the HUD display a stamina bar for either character, updated in real time?	Run program	The HUD displays coloured bars which display either characters' stamina in real time.	
4C	Does the HUD dynamically display the player's cooldown as a bar?	Run program	The HUD displays the player's cooldown value, in real time, showing only it when necessary (cooldown is not zero).	
4D	Does the HUD display a play/pause button is the top left corner for the user?	Run program	A play/pause button is displayed in the appropriate region. On clicking it the icon/sprite changes to reflect the state, with the game being paused/resumed.	
4E	Do all of the bars show relevant information which is consistently updated?	Run program	Yes.	

Stage 5 - Start Menu

Test ID	Test	Test Data	Outcome	Actual Outcome
5A	Does the game start with a menu?	Run program	The game loads with a menu with the option to start the game.	
5B	Does this start menu include a difficulty slider?	Run program	Yes.	
5C	Does the difficulty slider accurately update a value which affects the difficulty of the game?	Run program	Yes.	
5D	Does the start button start the game?	Run program	On click, the start button begins the game with both the player and enemy set up?	

Stage 6 - Better Attacks

Test ID	Test	Test Data	Outcome	Actual Outcome
6A	Do attacks have a value for knockback	Run program	Yes.	

Test ID	Test	Test Data	Outcome	Actual Outcome
6B	as well as damage? Is knockback applied accurately to either character when necessary?	Run program	Yes.	

Stage 7 - Condition Checks

Test ID	Test	Test Data	Outcome	Actual Outcome
7A	Does the player win when the enemy's health is 0?	Run program	The player wins the game and an appropriate screen is displayed to convey this.	
7B	Does the player lose the game when their health is 0?	Run program	The enemy wins and a message is shown to display this.	
7C	Can the player attack when their stamina is 0?	Run program	The player cannot attack until their stamina is sufficiently regenerated.	
7D	Does the stamina amount regenerate exponentially after a period of inactivity?	Run program	Yes.	
7E	Can the player use an attack when it is on Cooldown ?	Run program	No, they must wait until the Cooldown	

Test ID	Test	Test Data	Outcome	Actual Outcome
7F	Is a character removed when their health is 0?	Run program	timer has ended. The character is removed from the scene.	

Stage 8 - Better Enemies

Test ID	Test	Test Data	Outcome	Actual Outcome
8A	Does the enemy load with random strength and knockback in accordance with difficulty?	Run program	Yes.	
8B	Does the enemy have a random size in accordance with difficulty?	Run program	Yes.	

Final Tests

Test ID	Test	Test Data	Purpose	Actual Outcome
F1	Player Enemy collision	Run program	Tests if the player and enemy are capable of colliding.	

Test ID	Test	Test Data	Purpose	Actual Outcome
F2	Appropriate knockback levels for attacks	Run program	Tests if the level of knockback applied is suitable for player and enemy.	
F3	Appropriate health values in correspondence with damage	Run program	Tests how long the average game could last and is this is consistent with my analysis and planning.	
F4	The player dying ends the game	Run program	To check if there is an appropriate response to inform the player that they have lost.	
F5	The start screen starts the game correctly	Run program	To ensure that random values for the enemy are correctly created with reference to the difficulty slider.	
F6	The gravity of the game works to achieve a pseudo realism	Run program	To see how the game feels to play for a player. A lower gravity could make for a slower	

Test ID	Test	Test Data	Purpose	Actual Outcome
			game, with more time spent in the air.	

Post Development Plan

Once each of my stages have been implemented, and I have alpha tested, I would like to ask some of my friends and peers to beta test my game. In order for this to be effective and a good use of time, I will put together series of questions to ask them once they have played the game on each difficulty setting.

My questions should aim to cover information about the following:

- User friendliness
- Intuitive design
- Likeability of the design
- Difficulty of each setting
- How long each difficulty took them
- The usability of the controls
- Did the game perform well (high frames per second, etc)

In light of this, my questions will probably look something like this:

1. On a scale of 1 to 10, how user friendly is the game?
2. On a scale of 1 to 10, how intuitive would you describe the design?
3. Did you like the design?
4. On a scale of 1 to 10, how hard was the hardest setting?
5. On a scale of 1 to 10, how hard was the default setting?
6. On a scale of 1 to 10, how hard was the easiest setting?
7. How long did the hardest difficulty take you?
8. How long did the default difficulty take you?
9. How long did the easiest difficulty take you?
10. Did you find the controls to be easy to learn and use?
11. Would you say the game performed well on your system?
12. Did you encounter any bugs/glitches?

My hope is that the beta testers will enjoy the game and give constructive feedback to work on before it is finished.

This questionnaire will most likely be sent via Microsoft Forms, due to the Microsoft Office Suite being available to my college. This would make it the easiest form of communication with integrated data graphics.

Stage 1

Description and Goals

Stage 1 involves creating the player with adequate movement abilities. This extends to attacks, which for now just involves changing the sprite.

To Do List

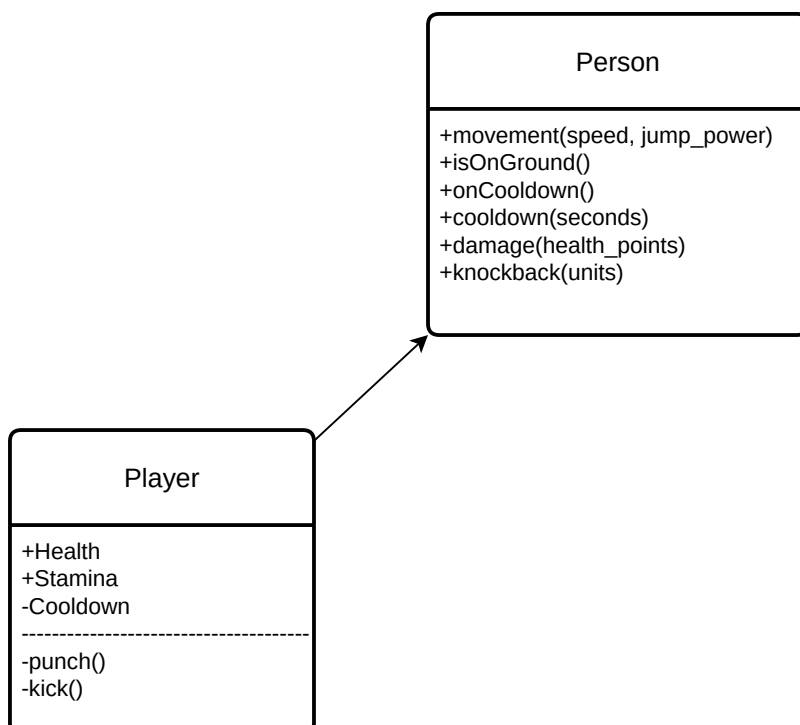
Features

- ✓ Player
 - ✓ Can move with keyboard input (left, right, jump)
 - ✓ Can attack with keyboard input
 - ✓ Player returns to ground after jumping

Tests

- ✓ Player sprite displayed?
- ✓ Accurate movement?
- ✓ Attack animation displayed?
- ✓ Gravity?
- ✓ Health Values?

Class Diagram

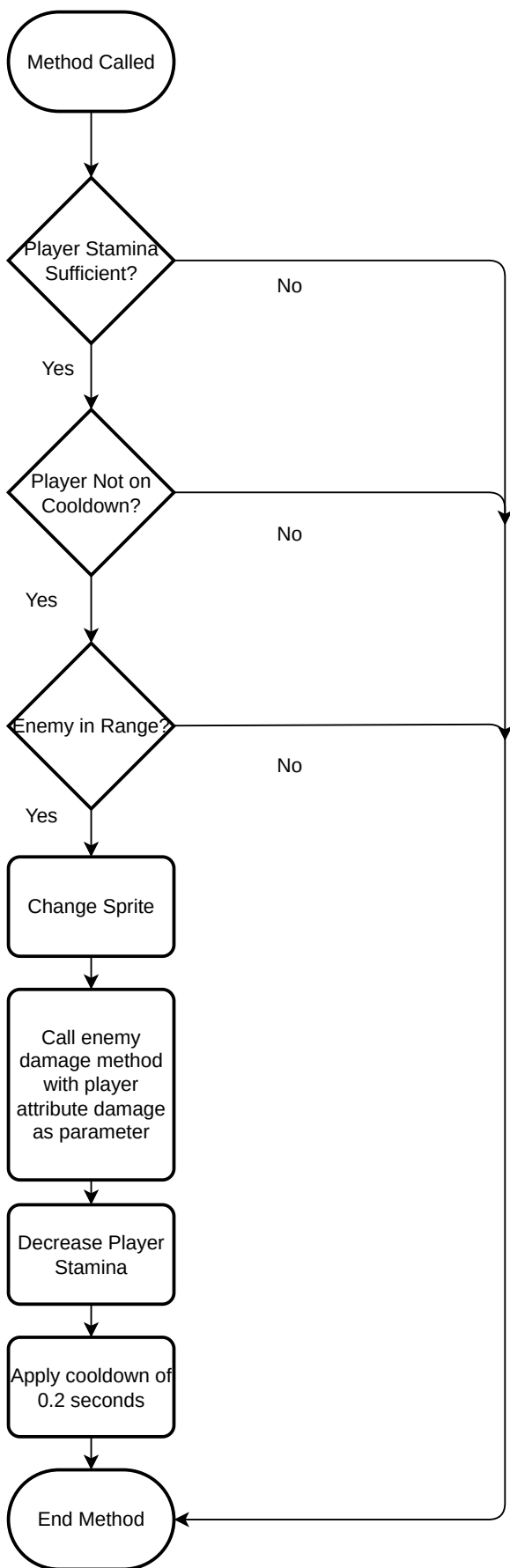


This class diagram holds all of the attributes and methods for my player. I have produced this because it

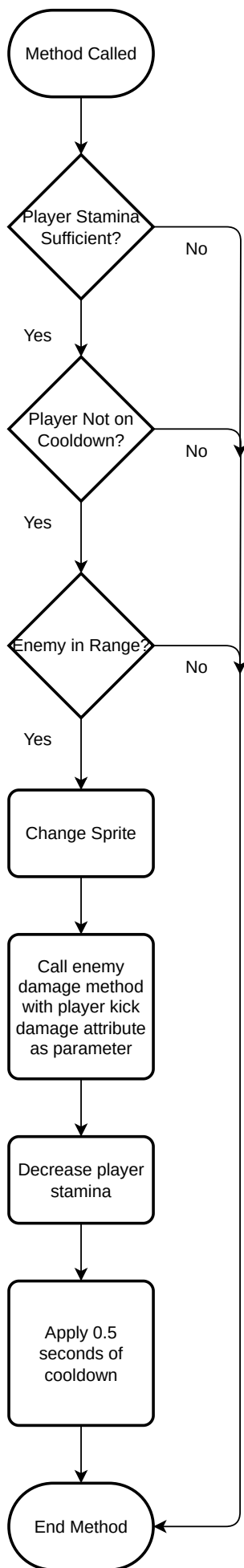
will be incredibly useful to know all of the methods which I will need to implement and attributes to declare. Please note that this would not include all variables. Ideally, I would have some declared like constants to make it easier to modify data such as horizontal acceleration.

Methods

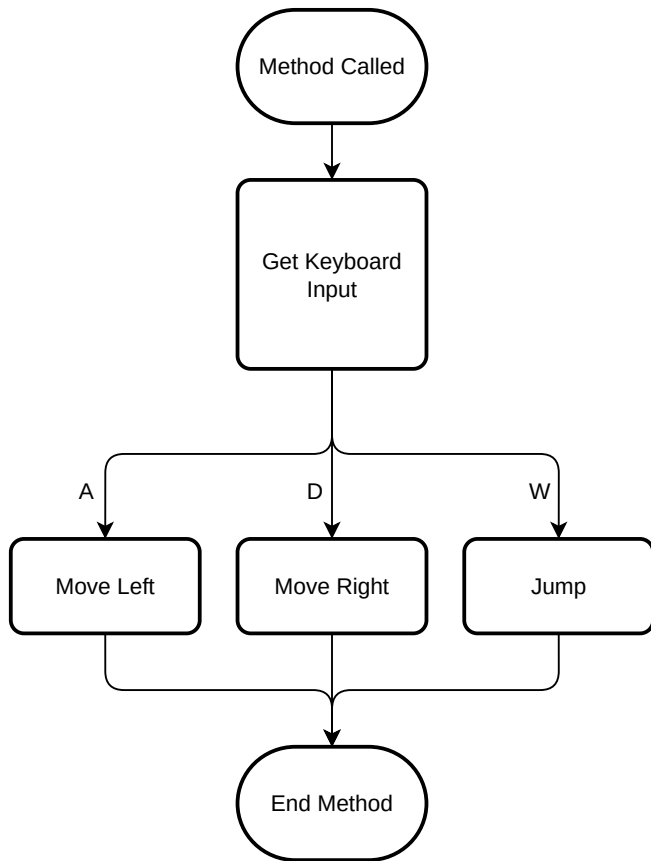
punch()



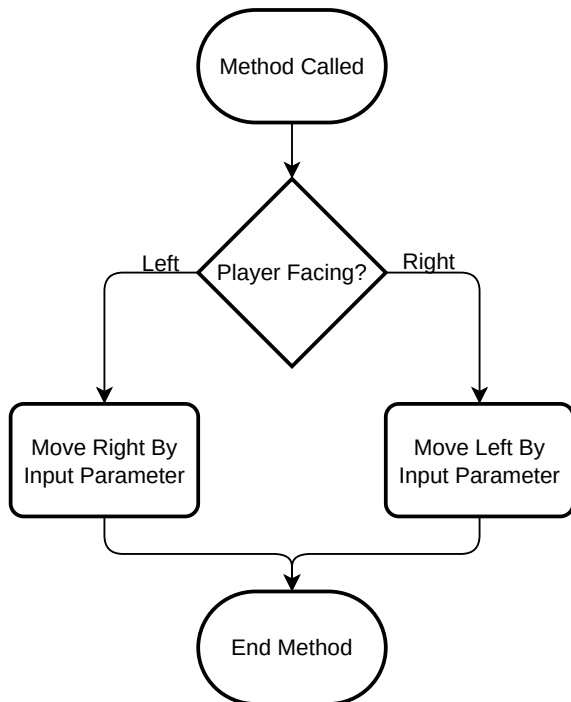
kick()



movement()



knockback()



damage()

```
function damage(int hp) then
    this.health = this.health - hp;
end function
```

Attributes

Health

Attribute to control the character's health. When `player.health == 0`, game over.

Stamina

Controls whether or not the player is allowed to attack.

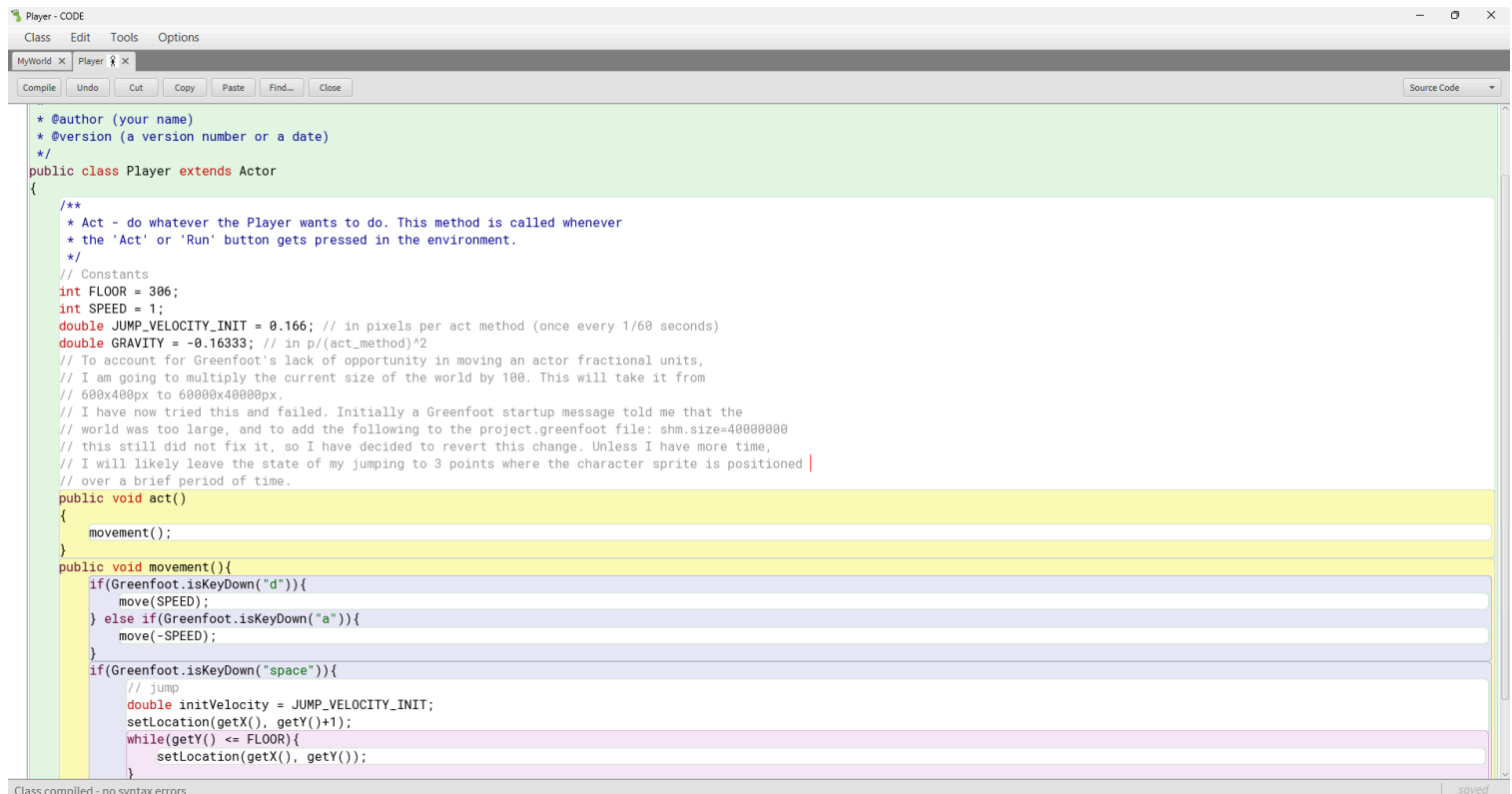
Cooldown

Same as `stamina` but with more direct consequences. Stamina is designed to limit and restrict against attacking relentlessly, whilst `cooldown` should stop the player from being able to attack every frame (too fast).

Notes

[references > How to round to an integer](#)

Pixel Rounding Error

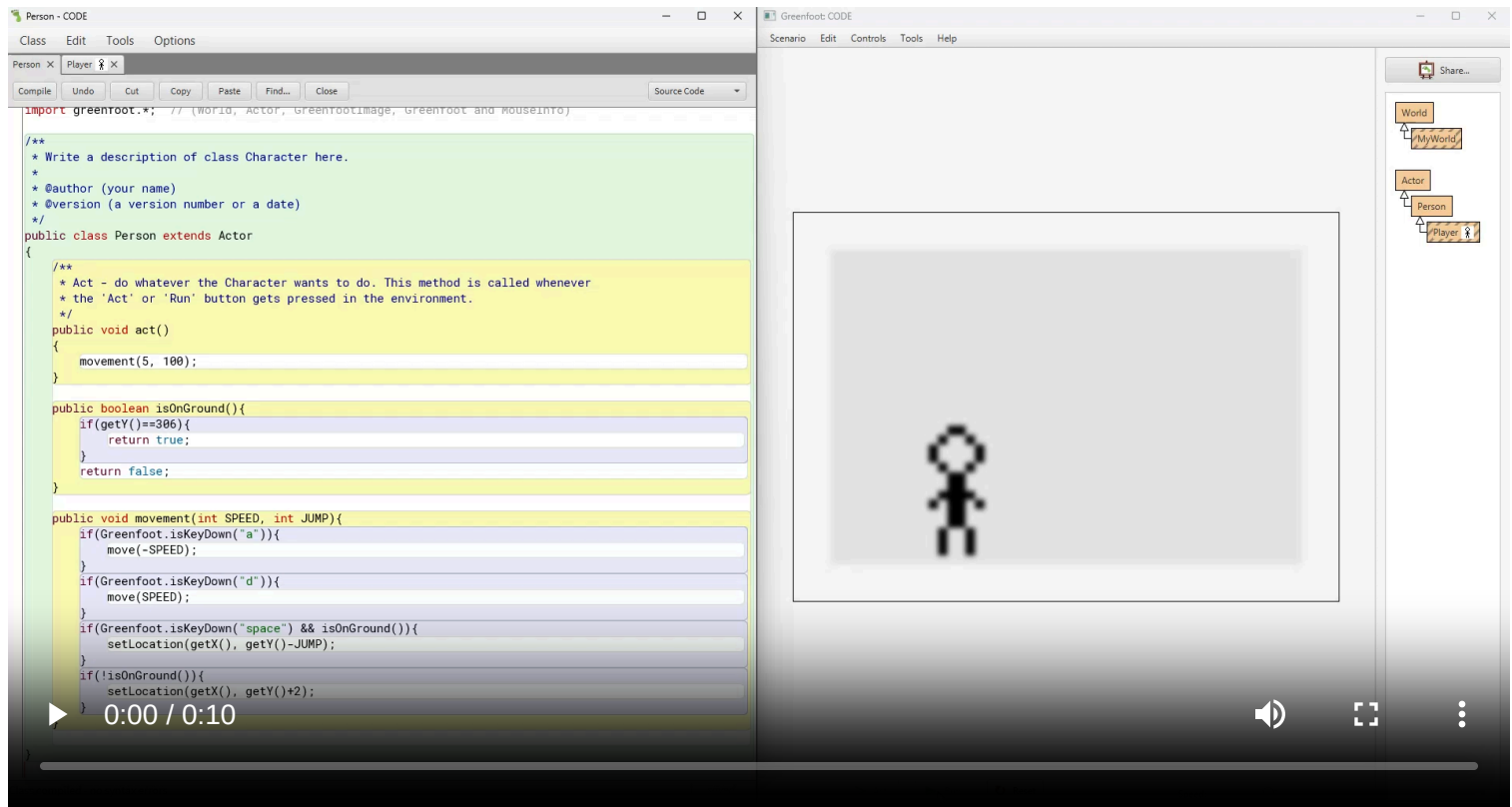


```
Player - CODE
Class Edit Tools Options
MyWorld x Player x
Compile Undo Cut Copy Paste Find... Close Source Code
/**
 * @author (your name)
 * @version (a version number or a date)
 */
public class Player extends Actor
{
    /**
     * Act - do whatever the Player wants to do. This method is called whenever
     * the 'Act' or 'Run' button gets pressed in the environment.
     */
    // Constants
    int FLOOR = 306;
    int SPEED = 1;
    double JUMP_VELOCITY_INIT = 0.166; // in pixels per act method (once every 1/60 seconds)
    double GRAVITY = -0.16333; // in p/(act_method)^2
    // To account for Greenfoot's lack of opportunity in moving an actor fractional units,
    // I am going to multiply the current size of the world by 100. This will take it from
    // 600x400px to 60000x40000px.
    // I have now tried this and failed. Initially a Greenfoot startup message told me that the
    // world was too large, and to add the following to the project.greenfoot file: shm.size=40000000
    // this still did not fix it, so I have decided to revert this change. Unless I have more time,
    // I will likely leave the state of my jumping to 3 points where the character sprite is positioned |
    // over a brief period of time.
    public void act()
    {
        movement();
    }
    public void movement(){
        if(Greenfoot.isKeyDown("d")){
            move(SPEED);
        } else if(Greenfoot.isKeyDown("a")){
            move(-SPEED);
        }
        if(Greenfoot.isKeyDown("space")){
            // jump
            double initVelocity = JUMP_VELOCITY_INIT;
            setLocation(getX(), getY()+1);
            while(getY() <= FLOOR){
                setLocation(getX(), getY());
            }
        }
    }
}
```

This is the first major error which I encountered. I was overambitious in my designing of a jump method, and wanted it to be physically accurate, using equations of motion. The problem with this was that Greenfoot would not have been able to move an object a fractional number of pixels, meaning that my

JUMP_VELOCITY_INIT and GRAVITY variables would not be viable, even if I were to round them. I tried to solve this by multiplying the number of pixels in the world by 100, in order to do the same to the variables and reduce the rounding errors. This would not work due to Greenfoot's own limit on pixels. This led to me having to scrap this idea.

Moving and Jumping Proof



In this video (moving_and_jumping.mp4), we can see the player moving and jumping, through the inherited move method from the parent class `Person`. The jumping is quite janky because of the use of `setLocation(getX(), getY()-JUMP)`. I tried to incorporate a gradual increase such as

```
while(getY() >= FLOOR - JUMP){
    setLocation(getX(), getY()-5);
}
while(getY() < FLOOR){
    setLocation(getX(), getY()+5);
}
```

But this became a problem with Greenfoot. I am still unsure why, but it would cause the program to crash. The current solution is now a compromise.

Old Cooldown Method

```

56 // setters
57 public void cooldown(double t){ // t in seconds
58     this.cooldown = t*60; // greenfoot tick speed is 60 per second, this allows t to be passed as seconds
59     for(int c = this.cooldown; c>=0; c--){ // uses this.cooldown to reference object's value
60         this.cooldown--; // eventually sets cooldown back to 0
61     }
62 }
63
64

```

■ Type mismatch: cannot co

This method was the old `cooldown` method before the new one

(

```

import java.math.RoundingMode;
import java.text.DecimalFormat;

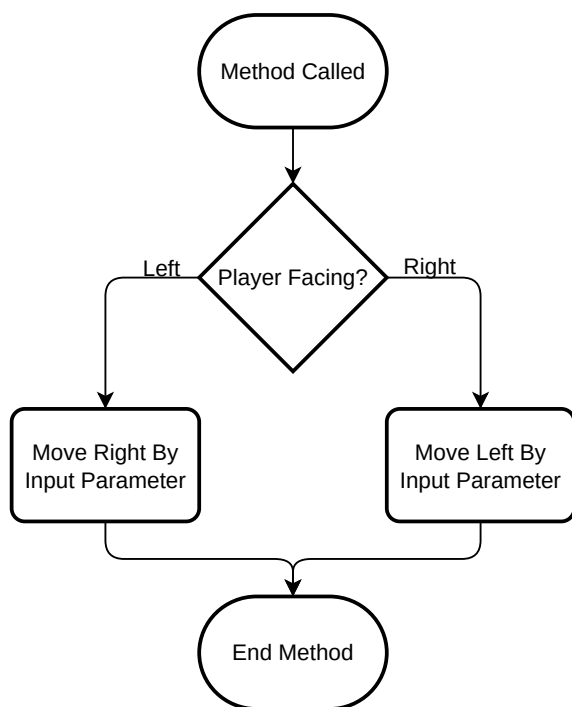
public void cooldown(double t){
    double unrounded_cooldown = t*60;
    DecimalFormat df = new DecimalFormat("###"); // where unused digits are 0
    df.setRoundingMode(RoundingMode.CEILING); // rounds up from t*60
    this.cooldown = df.format(unrounded_cooldown); // sets the cooldown to
appropriate integer
    for(int c = this.cooldown; c>=0; c--){ // same logic as if it were an int
        this.cooldown--;
    }
}

```

). As can be seen, `this.cooldown` is incorrectly assigned as `t*60`. This is because I had forgotten my initial plan to have the punch cooldown as 0.2 seconds. This adjustment was the reason which I had to change to an algorithm with explicit rounding.

With regards to Knockback

I have at present chosen not to implement the full `Knockback` method as was planned in my flowchart.



This is because I think that finding out which way the player was facing (especially in Greenfoot), would

take quite a lot of time. Therefore, I have at present left this either for the future or permanence. The current method stands at:

```
public void knockback(int k){  
    move(-k);  
}
```

This takes a heuristic approach, assuming that the player will always be on the left, attacking right. It is far more concise than the flow chart demands it be, which is why I have decided that it is currently the best option.

Fixing A Rounding Error

In the previous fix for the `cooldown` method, I employed the libraries `math.RoundingMode` and `text.DecimalFormat`. This is shown in [Old Cooldown Method](#). I have since dealt with an error produced by the `df.format()` call. As it happens, this method returns a string type, which caused a problem with `cooldown`, being defined as an integer. I fixed this by adding

```
Integer.parseInt(df.format(target));
```

thus converting the "string" to the desired integer.

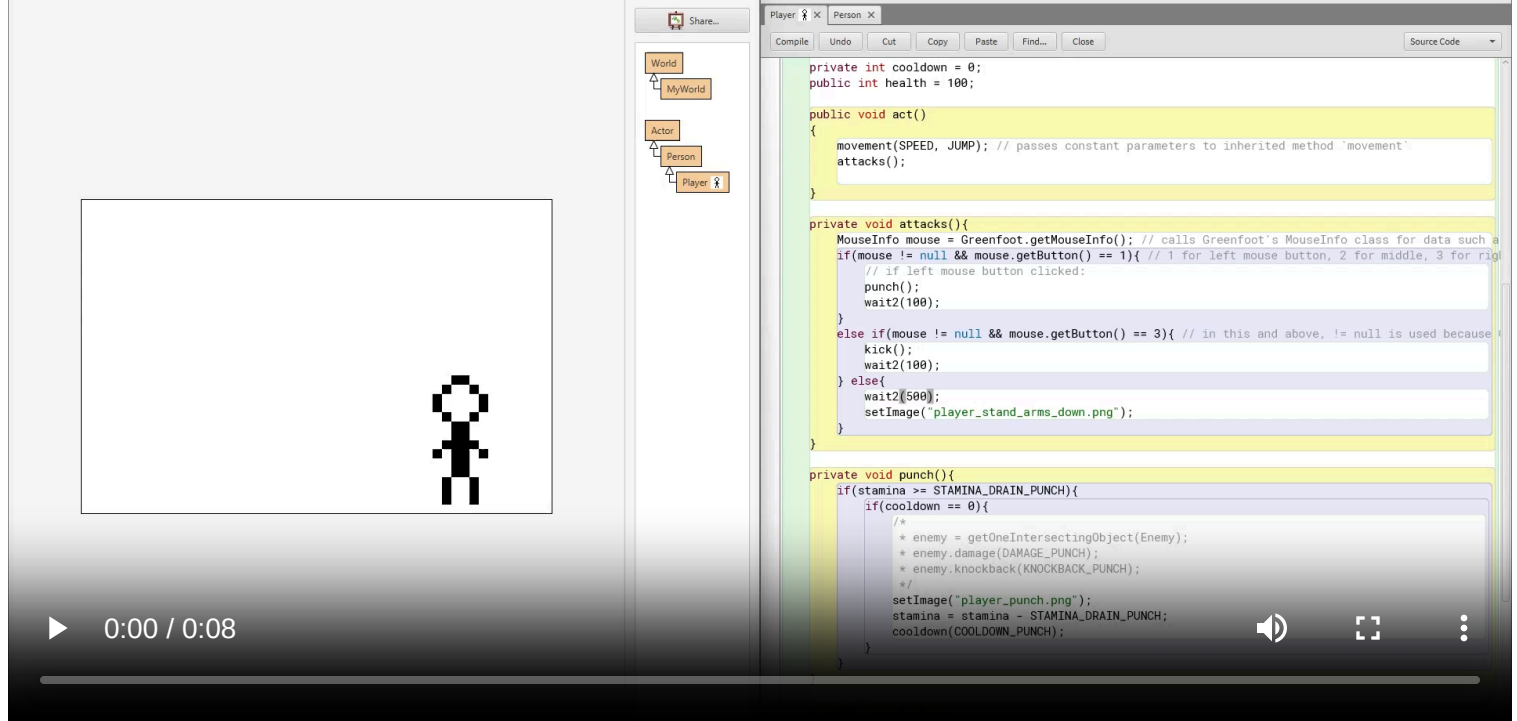
Round() Method

Whilst implementing a `wait2()` method (further explained later), I was going to reuse the rounding code from my `cooldown(double t)` method. Instead of doing this, I optimised the code by introducing a new `round(double target)` function.

```
public int round(double target){  
    DecimalFormat df = new DecimalFormat('###');  
    df.setRoundingMode(RoundingMode.CEILING);  
    return Integer.parseInt(df.format(target));  
}
```

Sprite Setting

Whilst I had technically completed my test plan, I had not accounted for some of the problems which I may have encountered. One of them is that I had not checked that the player's sprite would revert back to the default after an attack sprite is displayed.



(attack_sprite_timing_error.mp4)

```

private void attacks(){
    MouseInfo mouse = Greenfoot.getMouseInfo(); // calls Greenfoot's MouseInfo class for data such as
    if(mouse != null && mouse.getButton() == 1){ // 1 for left mouse button, 2 for middle, 3 for right
        // if left mouse button clicked:
        punch();
        wait2(100);
        setImage("player_stand_arms_down.png");
    }
    if(mouse != null && mouse.getButton() == 3){ // in this and above, != null is used because
        kick();
        wait2(1);
        setImage("player_stand_arms_down.png");
    }
}

```

I thought that it would be an easy fix like in the above image. Sadly this does not work. I was able to trace the point of vulnerability to the `wait2()` which I then tried to fix.

```

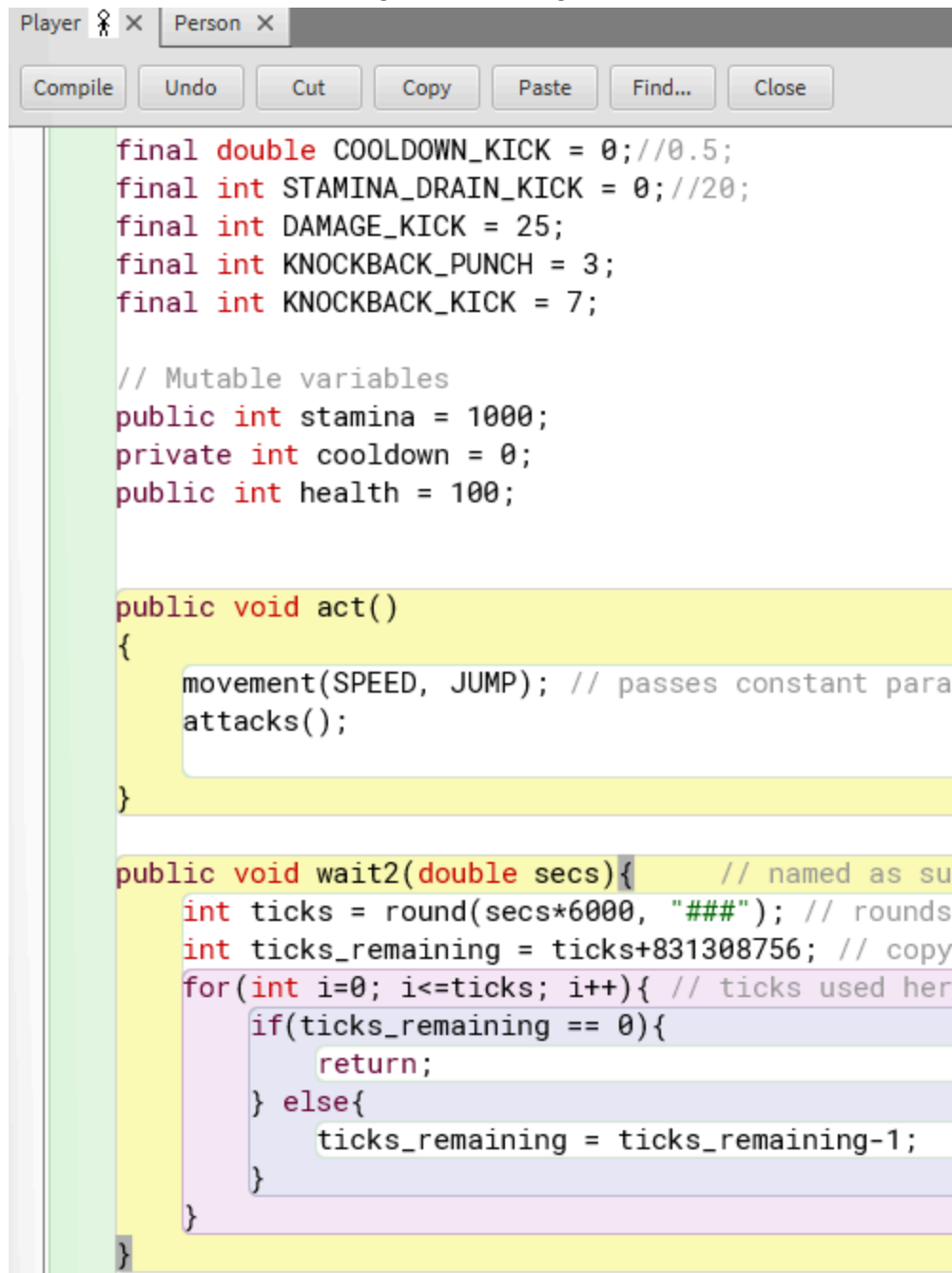
public void wait2(double secs){ // named as such because wait() is a method in the Java API
    int ticks = round(secs*6000, "###"); // rounds value for how many greenfoot ticks to wait
    int ticks_remaining = ticks+831308756; // copy of ticks to iterate down to 0
    for(int i=0; i<=ticks; i++){ // ticks used here because ticks_remaining is not updated yet
        if(ticks_remaining == 0){
            return;
        } else{
            ticks_remaining = ticks_remaining-1;
        }
    }
}

```

As you can see in

```
int ticks_remaining = ticks+831308756;
```

I tested the function by iterating through arbitrarily large numbers. This is because the core idea of the function is that if each second runs 60 frames, then 60 frames can be populated with a 'spinner loop' to accurately wait 1 second. My test would have accounted for a minimum of 13855145.93333333... seconds. Unfortunately this did not work so I theorised that the error stemmed from the fact that it was an inherited method and so might be running in a different thread.



The screenshot shows a Java IDE window titled 'Player' with a 'Person' tab. The code defines a Player class with several final constants, mutable variables, and two methods: act() and wait2(). The wait2() method contains a loop that decrements ticks_remaining until it reaches 0, effectively waiting for a specified number of seconds.

```
final double COOLDOWN_KICK = 0;//0.5;
final int STAMINA_DRAIN_KICK = 0;//20;
final int DAMAGE_KICK = 25;
final int KNOCKBACK_PUNCH = 3;
final int KNOCKBACK_KICK = 7;

// Mutable variables
public int stamina = 1000;
private int cooldown = 0;
public int health = 100;

public void act()
{
    movement(SPEED, JUMP); // passes constant para
    attacks();
}

public void wait2(double secs){ // named as su
    int ticks = round(secs*6000, "###"); // rounds
    int ticks_remaining = ticks+831308756; // copy
    for(int i=0; i<=ticks; i++){ // ticks used her
        if(ticks_remaining == 0){
            return;
        } else{
            ticks_remaining = ticks_remaining-1;
        }
    }
}
```

Unfortunately this was to the same effect, even with slight modification as below:

```
public void wait2(int secs){    // named as such because wait() is already
    //int ticks = round(secs*6000, "###"); // rounds value for how many gree
    int ticks_remaining = 831308756; // copy of ticks to iterate down
    /*for(int i=0; i<=secs*60; i++){ // ticks used here because ticks_remain
        if(ticks_remaining == 0){
            return;
        } else{
            ticks_remaining = ticks_remaining-1;
        }
    }*/
    while(ticks_remaining > 0){
        ticks_remaining--;
    }
}
```

Fixing the Sprite Setting

Today (01.10.26) I fixed the sprite setting issue. I realised that it was caused by the fact that it was the **main act method** which called 60 times per second. This in conjunction with my broken `wait2()` function led to me implementing a manual timer held in the Player act method.

Player  X Person X

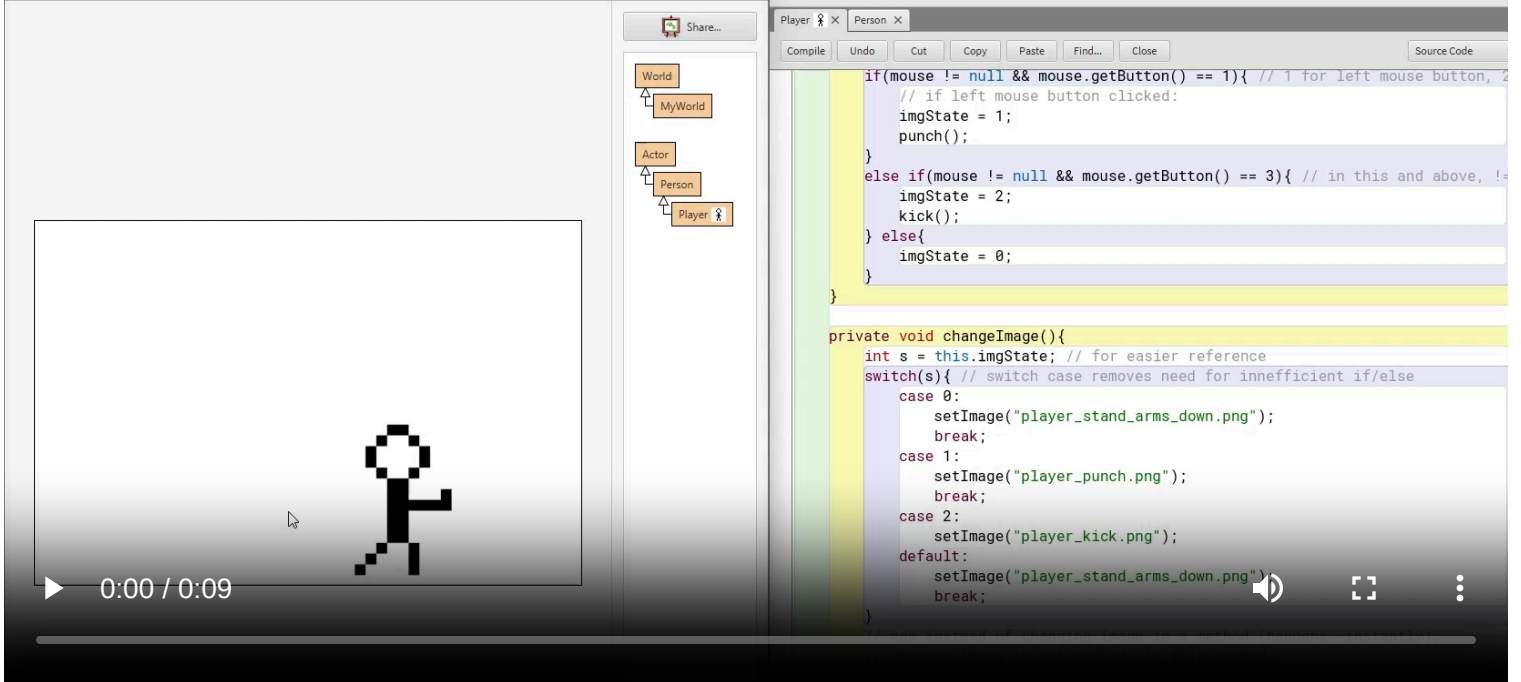
Compile Undo Cut Copy Paste Find... Close Source C

```
final int STAMINA_DRAIN_PUNCH = 10;
final double COOLDOWN_PUNCH = 0.2;
final double COOLDOWN_KICK = 0.5;
final int STAMINA_DRAIN_KICK = 20;
final int DAMAGE_KICK = 25;
final int KNOCKBACK_PUNCH = 3;
final int KNOCKBACK_KICK = 7;

// Mutable variables
public int stamina = 100;
private int cooldown = 0;
public int health = 100;
private int imgState = 0; // 0 as default, 1 for punch, 2 for kick
private int imgCountdown = 0; /* theoretically should limit time for image
change. Each act method this will increment and thus change the image for
sprite.
*/
public void act()
{
    // variables
    imgCountdown++;
    if(cooldown > 0){
        cooldown--;
    }
    if(stamina < STAMINA_MAX){
        stamina++;
    }
    // image changes
    if(imgCountdown == 30){
        changeImage();
        imgCountdown = 0;
    }
}
```

The fact that this solution works is making me consider a solution to the initial wait problem. As a result, in my next stage I might include a similar wait method with the decrementing number is stored in a super class outside of the method. It could be added to with the act method responsible for decreasing the cooldown value once per tick.

Proof:



(fixed_sprite_changes.mp4)